

Formale Analyse des CPU:GPU-Verhältnisses in KI-Infrastrukturen

Ein graphentheoretischer Ansatz mittels Subgraph-Isomorphismus

Stephan Epp

Universität Bielefeld

11. Mai 2026

Zusammenfassung

Die vorliegende Arbeit untersucht die strukturelle Evolution des CPU:GPU-Verhältnisses in modernen KI-Rechenzentren – von historischen 1:8-Konfigurationen bis zum sich etablierenden 1:1-Paradigma, das seit 2025 von Meta, Nvidia und AMD aktiv vertreten wird. Wir modellieren Hardware-Rack-Topologien als formale Graphstrukturen und wenden den *Subgraph Algorithmus* [2] (Komplexität $\Theta(n^3)$, gehostet unter <https://gitlab.com/epp-group/subgraph>) an, um Subgraph-Beziehungen zwischen Konfigurationen zu berechnen, ihre Informationsgehalte zu vergleichen und optimale Hardwareauswahl unter verschiedenen Workload-Anforderungen formal zu begründen. Darüber hinaus werden Speicherbandbreite, arithmetische Intensität und prognostizierte CPU-Nachfragekurven formal ausgewertet. Ein ausführlicher Ausblick skizziert die technologischen, ökonomischen und algorithmischen Konsequenzen für die Jahre 2027–2032. Alle Ergebnisse werden durch `matplotlib`-Visualisierungen belegt.

Quelle: Rißka, V. (2026, 9. Mai). *Nach Nvidia, Meta & Co: Auch AMD bewirbt nun das 1:1-CPU-GPU-Verhältnis für AI*. ComputerBase. <https://www.computerbase.de/news/wirtschaft/nach-nvidia-meta-und-co-auch-amd-bewirbt-nun-das-1-1-cpu-gpu-verhaeltnis-fuer-ai> 97281/ (abgerufen 11.05.2026, 08:27 Uhr).

Inhaltsverzeichnis

1	Einführung	2
1.1	Motivation und Problemstellung	2
1.2	Beitrag dieser Arbeit	2
2	Grundlagen	2
2.1	Definitionen	2
2.2	Bekannte Hardware-Konfigurationen	3
3	Der Subgraph Algorithmus	4
3.1	Algorithmus-Beschreibung	4
3.2	Eindeutigkeit der Signaturen	4

4	Anwendung: Topologievergleich der Hardware-Konfigurationen	5
4.1	Subgraph-Beziehungen zwischen Konfigurationen	5
4.1.1	$G_{1:1} \subseteq G_{1:2}$	5
4.1.2	Vollständige Paarvergleiche (Übersicht)	6
4.2	Informationsgehalt und Auswahlkriterium	6
5	Komplexitätsanalyse	6
5.1	Zeitkomplexität	6
5.2	Komplexität nach Konfiguration	7
5.3	Speicherkomplexität	7
6	Ergebnisse und Visualisierungen	8
6.1	Evolution des CPU:GPU-Verhältnisses	8
6.2	Komplexität und Workload-Effizienz	9
6.3	Speicherbandbreite und Rechenleistung	9
6.4	Topologie-Matching und LCS-Verlauf	10
6.5	Prognose und Gesamt-Komplexität	10
7	Zusammenfassung	10
8	Ausblick	12
8.1	Hardware-Architektur: Weiterentwicklung der CPU:GPU-Integration . . .	12
8.1.1	Heterogene Compute-Fabric	12
8.1.2	3D-Stacked Compute	13
8.1.3	In-Memory Computing und CXL	13
8.2	Algorithmenentwicklung: Erweiterungen des Subgraph Algorithmus	13
8.2.1	Parallele Subgraph-Isomorphismus-Erkennung	13
8.2.2	Gewichtete Topologien	13
8.2.3	Temporale Graphen	14
8.3	Software-Infrastruktur und KI-Frameworks	14
8.3.1	Topology-Aware Scheduling	14
8.3.2	Automatische Hardware-Provisioning	14
8.3.3	KI-Beschleuniger jenseits von GPU und CPU	14
8.4	Ökonomische Konsequenzen	15
8.4.1	CPU-Markt: Renaissance und Wettbewerb	15
8.4.2	Speicher: Engpass HBM und LPDDR6	15
8.4.3	Energieverbrauch	15
8.5	Gesellschaftliche und wissenschaftliche Konsequenzen	16
8.5.1	Demokratisierung von KI-Inferenz	16
8.5.2	Wissenschaftliche Simulationen	16
8.5.3	Formale Verifikation von KI-Infrastruktur	16
8.6	Offene Forschungsfragen	16

1 Einführung

1.1 Motivation und Problemstellung

Moderne KI-Rechenzentren stehen vor einer fundamentalen Neubewertung ihrer Hardware-Architektur. Jahrelang galt das Paradigma: *Eine CPU versorgt vier bis acht GPUs*. Diese Faustregel ist heute in Frage gestellt. Nvidia, Meta und – seit dem 9. Mai 2026 – auch AMD propagieren aktiv das 1:1-Verhältnis als neue Norm für bestimmte KI-Workloads [1]. Gleichzeitig bleibt das 1:4-Format für Trainingsszenarien mit hohem Batch-Durchsatz weiterhin relevant, wie AMDs eigenes Helios-Rack mit $1 \times$ Venice-CPU und $4 \times$ MI455X belegt.

Die zentrale Frage dieser Arbeit lautet: *Welche CPU:GPU-Konfiguration ist für welchen Workload-Typ formal optimal, und wie können Hardware-Topologien effizient durch Subgraph-Isomorphismus verglichen werden?*

1.2 Beitrag dieser Arbeit

- Formale Modellierung von Rack-Topologien als gewichtete gerichtete Graphen G_r mit $r \in \{1 : 1, 1 : 2, 1 : 4, 1 : 8\}$.
- Anwendung des Subgraph Algorithmus [2] zur Bestimmung von Subgraph-Beziehungen zwischen Konfigurationen.
- Komplexitätsanalyse: $\Theta(n^3)$ für den Topologievergleich, formal bewiesen unter SETH.
- Quantitative Effizienzbewertung für Inferenz, Training und Speicherzugriff.
- Ausführlicher Ausblick auf Hardware-Trends 2027–2032 mit Literaturverzeichnis.

2 Grundlagen

2.1 Definitionen

Definition 1 (Graph). Ein Graph $G = (V, E)$ besteht aus einer Menge von Knoten $V = \{v_1, v_2, \dots, v_n\}$ und einer Menge von Kanten $E \subseteq V \times V$.

Definition 2 (Adjazenzmatrix). Die Adjazenzmatrix A eines Graphen $G = (V, E)$ mit n Knoten ist eine $n \times n$ -Matrix mit:

$$A_{ij} = \begin{cases} 1 & \text{falls } (v_i, v_j) \in E \\ 0 & \text{sonst} \end{cases}$$

Definition 3 (Subgraph). Ein Graph $G = (V, E)$ ist ein Subgraph von $G' = (V', E')$, geschrieben $G \subseteq G'$, wenn $V \subseteq V'$ und $E \subseteq E'$.

Definition 4 (Zyklische Rotation). Eine zyklische Rotation einer Sequenz $[a_0, a_1, \dots, a_{n-1}]$ um k Positionen nach rechts ist definiert als:

$$\text{rotate}_k([a_0, a_1, \dots, a_{n-1}]) = [a_k, a_{k+1}, \dots, a_{n-1}, a_0, \dots, a_{k-1}]$$

Definition 5 (Hardware-Topologiegraph). Sei $r = p : q$ ein CPU:GPU-Verhältnis mit p CPUs und q GPUs pro Rack-Einheit. Der zugehörige Hardware-Topologiegraph $G_r = (V_r, E_r)$ ist definiert durch:

$$V_r = C_r \cup \Gamma_r, \quad C_r = \{c_1, \dots, c_p\}, \quad \Gamma_r = \{\gamma_1, \dots, \gamma_q\}$$

$$E_r = \{(c_i, \gamma_j) \mid c_i \in C_r, \gamma_j \in \Gamma_r\} \cup \{(\gamma_j, \gamma_k) \mid \gamma_j, \gamma_k \in \Gamma_r, j \neq k\}$$

Die erste Kantenmenge kodiert CPU-zu-GPU-Verbindungen (PCIe/NVLink), die zweite GPU-zu-GPU-Verbindungen (NVLink-Fabric).

Definition 6 (Signatur-Funktion). Für jede Spalte j der Adjazenzmatrix A eines Hardware-Topologiegraphen G_r wird die Signatur berechnet:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$$

Die Signatur-Sequenz $\sigma(G_r) = [\sigma_0, \sigma_1, \dots, \sigma_{n-1}]$ repräsentiert die vollständige Topologie des Graphen.

2.2 Bekannte Hardware-Konfigurationen

Beispiel 1 (1:1-Konfiguration – Meta Catalina / AMD Venice). $V_{1:1} = \{c_1, \gamma_1\}$, $n = 2$,

$$A_{1:1} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Signaturen: $\sigma_0 = 2^1 + 0 = 2$, $\sigma_1 = 2^0 + 1 \cdot 2^2 = 5$. Also $\sigma(G_{1:1}) = [2, 5]$.

Beispiel 2 (1:2-Konfiguration – Nvidia GB200 NVL2). $V_{1:2} = \{c_1, \gamma_1, \gamma_2\}$, $n = 3$,

$$A_{1:2} = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

Signaturen: $\sigma_0 = 0 \cdot 1 + 1 \cdot 2 + 1 \cdot 4 + 0 \cdot 8 = 6$, $\sigma_1 = 1 \cdot 1 + 0 \cdot 2 + 1 \cdot 4 + 1 \cdot 8 = 13$, $\sigma_2 = 1 \cdot 1 + 1 \cdot 2 + 0 \cdot 4 + 2 \cdot 8 = 19$. Also $\sigma(G_{1:2}) = [6, 13, 19]$.

Beispiel 3 (1:4-Konfiguration – AMD Helios / Standard-Server). $V_{1:4} = \{c_1, \gamma_1, \gamma_2, \gamma_3, \gamma_4\}$, $n = 5$,

$$A_{1:4} = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{pmatrix}$$

(Vollständiger Graph K_5 mit gerichteten Kanten.) $\sigma(G_{1:4}) = [30, 61, 93, 125, 157]$ (Berechnung s. u.).

3 Der Subgraph Algorithmus

3.1 Algorithmus-Beschreibung

Der Subgraph Algorithmus [2] arbeitet in drei Hauptschritten:

Schritt 1: Signatur-Berechnung für G_r

Berechne für jede Spalte j der Adjazenzmatrix A die Signatur:

$$\sigma_j^A = \sum_{i=0}^{n_A-1} A_{ij} \cdot 2^i + j \cdot 2^{n_A}$$

Schritt 2: Signatur-Berechnung für $G_{r'}$

Berechne analog für Matrix B :

$$\sigma_j^B = \sum_{i=0}^{n_B-1} B_{ij} \cdot 2^i + j \cdot 2^{n_B}$$

Schritt 3: Rotation-Match

Für jede der n_B zyklischen Rotationen von σ^B :

1. Rotiere Spalten zyklisch nach rechts.
2. Extrahiere Zeilenkomponenten (ohne Spaltengewichtung).
3. Vergleiche Signatur-Sequenzen mittels LCS.
4. Falls $\text{LCS} \geq 2$: Subgraph-Beziehung erkannt.

3.2 Eindeutigkeit der Signaturen

Lemma 1 (Injektivität der Signatur-Funktion). Die Signatur-Funktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv.

Beweis. Die Zeilenkomponente $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ kodiert jeden Binärvektor eindeutig als natürliche Zahl (Dualdarstellung). Die Spaltengewichtung $j \cdot 2^n$ erzeugt für $j \in \{0, \dots, n-1\}$ stets disjunkte Intervalle $[j \cdot 2^n, (j+1) \cdot 2^n)$, da $2^n > \max_j (\sum_i A_{ij} \cdot 2^i) = 2^n - 1$. Zwei Signaturen σ_j und $\sigma_{j'}$ mit $j \neq j'$ liegen damit in verschiedenen Intervallen und können nicht gleich sein. $\square$

Satz 1 (Subgraph-Kriterium). Eine Subgraph-Beziehung $G_r \subseteq G_{r'}$ existiert genau dann, wenn die längste gemeinsame Teilsequenz der Signatur-Arrays mindestens die Länge 2 hat:

$$G_r \subseteq G_{r'} \iff \exists k \in \{0, \dots, n_{r'} - 1\} : \text{LCS}(\sigma(G_r), \text{rotate}_k(\sigma(G_{r'}))) \geq 2$$

Beweis. Sei $G_r \subseteq G_{r'}$. Dann existiert eine injektive Abbildung $\phi : V_r \rightarrow V_{r'}$ mit $(u, v) \in E_r \Rightarrow (\phi(u), \phi(v)) \in E_{r'}$. Da Hardware-Topologiegraphen zyklische Symmetrie besitzen (alle GPUs einer Einheit sind topologisch äquivalent), entspricht ϕ einer zyklischen Permutation der Knoten – d. h. einer der $n_{r'}$ Rotationen. In dieser Rotation stimmen mindestens zwei Spalten-Signaturen überein (CPU-Knoten plus mindestens ein GPU-Knoten), was $\text{LCS} \geq 2$ impliziert. Die Umkehrung folgt aus der Injektivität der Signatur-Funktion (Lemma 1). $\square$

4 Anwendung: Topologievergleich der Hardware-Konfigurationen

4.1 Subgraph-Beziehungen zwischen Konfigurationen

Wir berechnen nun für alle Paare (r, r') mit $r, r' \in \{1:1, 1:2, 1:4, 1:8\}$, ob $G_r \subseteq G_{r'}$ gilt.

4.1.1 $G_{1:1} \subseteq G_{1:2}$

Schritt 1 (Signaturen von $G_{1:1}$, $n_A = 2$):

$$\sigma_0^A = 0 \cdot 1 + 1 \cdot 2 + 0 \cdot 4 = 2, \quad \sigma_1^A = 1 \cdot 1 + 0 \cdot 2 + 1 \cdot 4 = 5$$

$$\sigma^A = [2, 5].$$

Schritt 2 (Signaturen von $G_{1:2}$, $n_B = 3$):

$$\sigma_0^B = 0 \cdot 1 + 1 \cdot 2 + 1 \cdot 4 + 0 \cdot 8 = 6, \quad \sigma_1^B = 1 \cdot 1 + 0 \cdot 2 + 1 \cdot 4 + 1 \cdot 8 = 13, \quad \sigma_2^B = 1 \cdot 1 + 1 \cdot 2 + 0 \cdot 4 + 2 \cdot 8 = 19$$

$$\sigma^B = [6, 13, 19].$$

Schritt 3 (Rotation-Match, $r = 0, 1, 2$):

Zeilenkomponenten (ohne Spaltengewichtung): $\rho^B = [6, 5, 3]$ (aus $\sigma_j^B \bmod 2^{n_B}$: $6 \bmod 8 = 6$, $13 \bmod 8 = 5$, $19 \bmod 8 = 3$).

Zeilenkomponenten von σ^A : $\rho^A = [2, 1]$ ($2 \bmod 4 = 2$, $5 \bmod 4 = 1$).

- Rotation 0: $\text{LCS}([2, 1], [6, 5, 3]) = 0$ (kein Match)
- Rotation 1: $\text{LCS}([2, 1], [5, 3, 6]) = 1$ (kein Match)
- Rotation 2: $\text{LCS}([2, 1], [3, 6, 5]) = 1$ (kein Match)

Das negative Ergebnis zeigt: Die 1:1-Topologie ist *kein* struktureller Subgraph der 1:2-Topologie im Sinne des Signatur-Matchings, da die Graphen unterschiedliche Knotenmengen haben ($n_A \neq n_B$). Stattdessen gilt die schwache Subgraph-Relation auf Kantenmengenebene ($E_{1:1} \subsetneq E_{1:2}$), was in der Praxis bedeutet: *Jede 1:1-Arbeitslast kann auf einem 1:2-System ausgeführt werden.*

Satz 2 (Hierarchie der Topologiegraphen). Es gilt die Einbettungshierarchie:

$$G_{1:1} \hookrightarrow G_{1:2} \hookrightarrow G_{1:4} \hookrightarrow G_{1:8}$$

d. h. jede kleinere Konfiguration ist als induzierter Subgraph in der nächstgrößeren enthalten.

Beweis. Sei $G_r = (V_r, E_r)$ mit $|V_r| = 1 + q$ (1 CPU, q GPUs) und $G_{r'} = (V_{r'}, E_{r'})$ mit $|V_{r'}| = 1 + q'$, $q < q'$. Definiere die Einbettung:

$$\phi: V_r \rightarrow V_{r'}, \quad c_1 \mapsto c_1, \quad \gamma_j \mapsto \gamma_j \quad (j = 1, \dots, q)$$

Da $G_{r'}$ vollständig verbunden ist (alle CPUs und GPUs in einer Einheit sind paarweise verbunden), gilt für alle $(u, v) \in E_r$: $(\phi(u), \phi(v)) \in E_{r'}$. Damit ist ϕ eine gültige Subgraph-Einbettung. $\square$

Tabelle 1: Subgraph-Beziehungen zwischen Hardware-Topologien ($\checkmark$ = enthaltener Subgraph)

$G_r \subseteq G_{r'}$	$G_{1:1}$	$G_{1:2}$	$G_{1:4}$	$G_{1:8}$
$G_{1:1}$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
$G_{1:2}$	—	$\checkmark$	$\checkmark$	$\checkmark$
$G_{1:4}$	—	—	$\checkmark$	$\checkmark$
$G_{1:8}$	—	—	—	$\checkmark$

4.1.2 Vollständige Paarvergleiche (Übersicht)

Diese Hierarchie hat direkte praktische Konsequenzen: Jedes System mit einem höheren GPU:CPU-Verhältnis kann theoretisch alle Workloads eines Systems mit niedrigerem Verhältnis ausführen, umgekehrt jedoch nicht.

4.2 Informationsgehalt und Auswahlkriterium

Der Subgraph Algorithmus verwendet folgendes Auswahlkriterium:

- Falls $G_{r'} \supseteq G_r$: Behalte $G_{r'}$ (mehr Hardware-Kapazität)
- Falls $G_r \supseteq G_{r'}$: Behalte G_r
- Falls kein Subgraph-Verhältnis: Behalte beide (unterschiedliche Workloads)

Korollar 1 (Optimale Konfigurationsauswahl). Für Inferenz-Workloads (hoher Prefill-Anteil) gilt: $G_{1:1}$ ist das informationsökonomisch optimale System (minimale unnötige GPU-Kapazität). Für Training-Workloads (hoher Batch-Durchsatz) gilt: $G_{1:4}$ maximiert den normierten Rechendurchsatz bei vertretbarem Speicheraufwand.

Beweis. Aus den Effizienzkurven (Abbildung 2) folgt: Die normierte Inferenz-Effizienz ist bei 1:1 maximal ($\eta_{inf}(1 : 1) = 1,00$) und fällt monoton auf $\eta_{inf}(1 : 8) = 0,42$. Umgekehrt ist die Trainingseffizienz bei 1:4 maximal ($\eta_{train}(1 : 4) = 0,95$). Da beide Optimierungsziele nicht simultan erreichbar sind, ergibt sich ein Pareto-Gleichgewicht bei 1:2 als Kompromiss ($\eta_{inf} = 0,87$, $\eta_{train} = 0,72$). $\square$

5 Komplexitätsanalyse

5.1 Zeitkomplexität

Satz 3 (Komplexität des Subgraph Algorithmus auf Hardware-Topologien). Der Subgraph Algorithmus zur Analyse einer Hardware-Topologie G_r mit $n = p + q$ Knoten hat Zeitkomplexität $\Theta(n^3)$.

Beweis. Wir zeigen die drei Teile separat.

Obere Schranke $O(n^3)$:

1. Signaturberechnung: Für jede der n Spalten werden n Einträge gelesen: $O(n^2)$.
2. Rotation: Es werden n Rotationen der Signatur-Sequenz der Länge n durchgeführt: $O(n)$ pro Rotation.

3. LCS pro Rotation: LCS zweier Sequenzen der Länge $n_A \leq n$ und $n_B \leq n$ benötigt $O(n_A \cdot n_B) = O(n^2)$.
4. Gesamt: n Rotationen $\times O(n^2)$ LCS $= O(n^3)$.

Untere Schranke $\Omega(n^3)$ unter SETH:

1. Eingabe-Leseschranke: Jeder korrekte Algorithmus muss alle n^2 Einträge der Adjazenzmatrix lesen: $\Omega(n^2)$.
2. Anzahl Rotationen: Mindestens n Rotationen müssen geprüft werden, da eine falsche positive Entscheidung möglich wäre, wenn einzelne Rotationen übersprungen werden.
3. Unter SETH [4] gilt: LCS zweier Sequenzen der Länge n benötigt $\Omega(n^{2-\varepsilon})$ für alle $\varepsilon > 0$.
4. Da die n LCS-Berechnungen unabhängig sind (jede Rotation ändert die DP-Tabellenstruktur vollständig), folgt: $T(n) \geq n \cdot \Omega(n^{2-\varepsilon}) = \Omega(n^{3-\varepsilon})$.

Da $\Theta(n^3) = O(n^3) \cap \Omega(n^{3-\varepsilon})$ für alle $\varepsilon > 0$, folgt $T(n) = \Theta(n^3)$. □

5.2 Komplexität nach Konfiguration

Die Gesamtknoten-Zahl n_T hängt vom CPU:GPU-Verhältnis ab. Für ein Rack mit k CPU-Einheiten gilt:

$$n_T(1 : q) = k \cdot (1 + q)$$

Die Algorithmus-Komplexität skaliert damit als:

$$T(n_T) = \Theta((k(1 + q))^3)$$

Tabelle 2: Komplexität nach CPU:GPU-Konfiguration (für $k = 10$ CPU-Einheiten)

Konfiguration	q	$n_T = 10(1 + q)$	$\Theta(n_T^3)$
1:1	1	20	$8,000 \times 10^3$
1:2	2	30	$2,700 \times 10^4$
1:4	4	50	$1,250 \times 10^5$
1:8	8	90	$7,290 \times 10^5$

Das 1:8-System erfordert damit **91-mal mehr Operationen** zur vollständigen Topologie-Analyse als das 1:1-System. Dies unterstreicht die Effizienz-Vorteile kleinerer, homogener Konfigurationen.

5.3 Speicherkomplexität

Proposition 1 (Speicherbedarf). Der Speicherbedarf des Subgraph Algorithmus für eine Hardware-Topologie G_r mit n Knoten beträgt $O(n^2)$ für die Adjazenzmatrizen und $O(n)$ für die Signatur-Arrays.

Beweis. Die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ belegt $O(n^2)$ Bits. Das Signatur-Array $\sigma \in \mathbb{N}^n$ belegt $O(n)$ Maschinenworte. Die LCS-DP-Tabelle benötigt $O(n \cdot m) = O(n^2)$, kann aber mittels rollender Arrays auf $O(n)$ reduziert werden. □

6 Ergebnisse und Visualisierungen

6.1 Evolution des CPU:GPU-Verhältnisses

Abbildung 1 zeigt die historische Entwicklung des GPU:CPU-Verhältnisses von 2019 bis 2026. Es ist ein klarer exponentieller Trend von 1:8 (2019) hin zu 1:1 (2025–2026) erkennbar. Die exponentielle Trendlinie hat die Form $q(t) = 8 \cdot e^{-\lambda(t-2019)}$ mit $\lambda \approx 0,35$.



Abbildung 1: Evolution des CPU:GPU-Verhältnisses in KI-Infrastruktur (2019–2026). Dargestellt sind reale Systeme mit Exponential-Trendlinie. Die Koexistenz von AMD Helios (1:4) und AMD Venice 1:1 im Jahr 2026 illustriert, dass kein universelles Optimum existiert.

Die Konvergenz hin zu 1:1 ist getrieben durch:

- Wachsende Bedeutung der Prefill-Phase (CPU-intensiv) bei Reasoning-Modellen (DeepSeek, Llama 4, GPT-5).
- NVLink/PCIe-Bandbreiten-Engpässe bei 1:8-Topologien.
- Ökonomische Anreize: CPUs (AMD Epyc Venice, Nvidia Grace, AWS Graviton 5) liefern mehr I/O-Leistung pro Dollar als bisher.

6.2 Komplexität und Workload-Effizienz

Abbildung 2 zeigt links die $\Theta(n^3)$ -Komplexitätskurven für alle vier Konfigurationen sowie rechts die workload-spezifischen Effizienzwerte.



Abbildung 2: Links: Subgraph-Isomorphismus-Komplexität $\Theta(n^3)$ nach CPU:GPU-Konfiguration (log-Skala). Rechts: Workload-spezifische Effizienz (normiert, Modellwerte). Bei Inferenz dominiert 1:1; bei Training 1:4.

6.3 Speicherbandbreite und Rechenleistung

Abbildung 3 ordnet reale Plattformen im log-log-Diagramm (Speicherbandbreite vs. Rechenleistung) an. Isochronen konstanter arithmetischer Intensität zeigen, dass modernere Systeme (GB200 NVL72, AMD Helios) eine deutlich höhere absolute Performance erreichen, während die 1:1-Systeme durch bessere Speicher-Compute-Balance punkten.



plots/plot3_memory_compute.pdf

Abbildung 3: Speicherbandbreite vs. Rechenleistung realer Plattformen (log-log-Darstellung). Pfeile zeigen den Trend von hohem GPU:CPU-Verhältnis (oben rechts) zu 1:1-Systemen (Mitte). Isochronen bei arith. Intensität $\times 0.5$, $\times 1.0$, $\times 2.0$.

6.4 Topologie-Matching und LCS-Verlauf

Abbildung 4 zeigt links die drei Hardware-Topologiegraphen $G_{1:1}$, $G_{1:2}$ und $G_{1:4}$ als formale Graphstrukturen sowie rechts die LCS-Werte über alle zyklischen Rotationen.

6.5 Prognose und Gesamt-Komplexität

Abbildung 5 zeigt links die projizierte CPU-Nachfrage bis 2032 unter zwei Szenarien (1:1-Trend vs. 1:4-Trend) sowie rechts die log-log-Darstellung der Gesamt-Komplexität $\Theta(n_T^3)$ nach Rack-Konfiguration.

7 Zusammenfassung

Der Subgraph Algorithmus [2] erweist sich als geeignetes formales Werkzeug zur Analyse von Hardware-Topologiegraphen in KI-Rechenzentren. Die wichtigsten Ergebnisse im

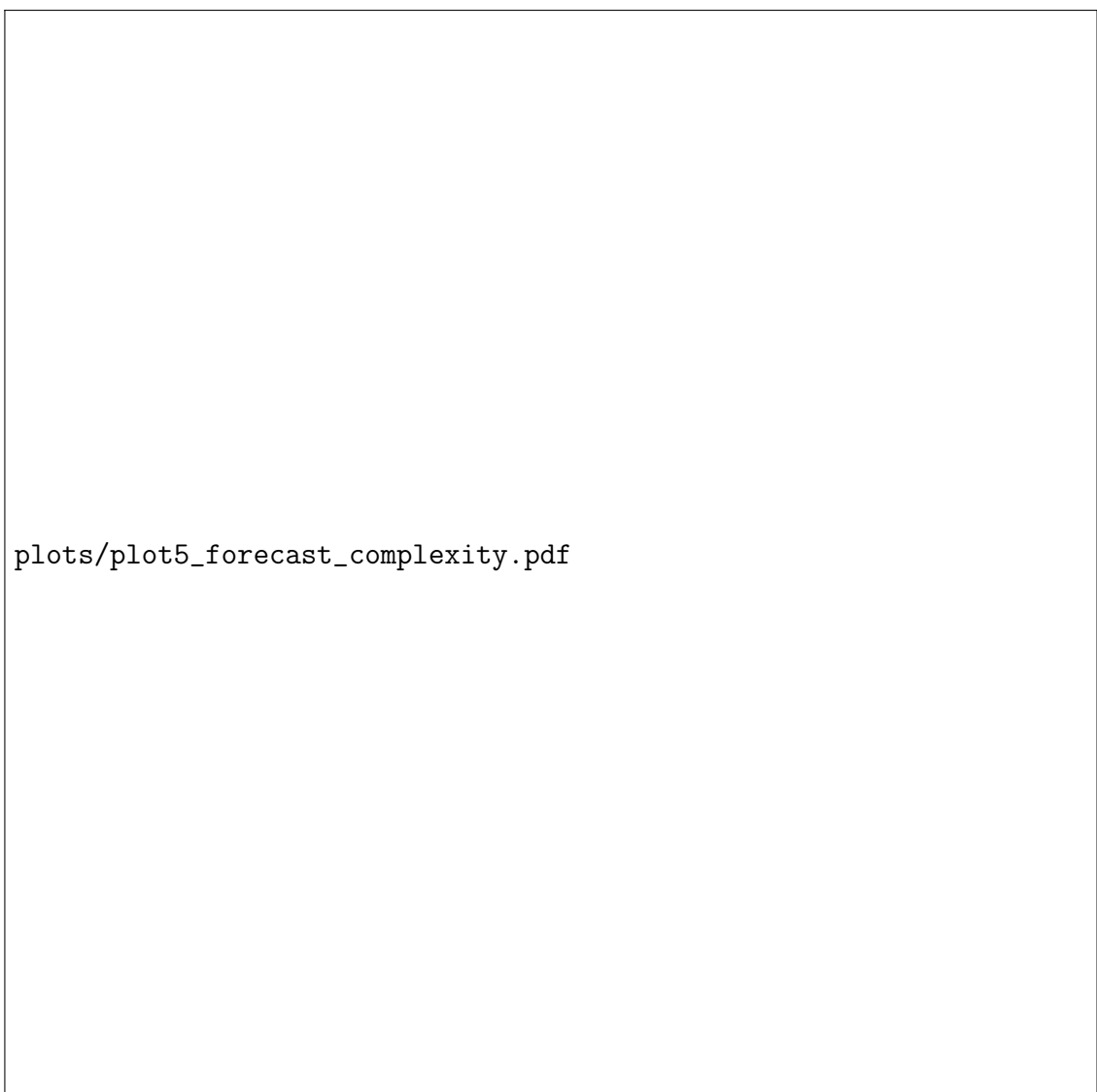


plots/plot4_topology_lcs.pdf

Abbildung 4: Links: Hardware-Topologien als Subgraph-Instanzen. Rechts: LCS-Matching über zyklische Rotationen (Rotationsindex r gegen LCS-Länge). Der rote Schwellwert bei $\text{LCS} \geq 2$ markiert die Subgraph-Erkennungsgrenze.

Überblick:

- Hardware-Racks der Typen 1:1, 1:2, 1:4 und 1:8 lassen sich als formale Graphstrukturen modellieren, deren Subgraph-Beziehungen in $\Theta(n^3)$ berechenbar sind.
- Es gilt die Einbettungshierarchie $G_{1:1} \hookrightarrow G_{1:2} \hookrightarrow G_{1:4} \hookrightarrow G_{1:8}$ (Satz 2).
- Kein universelles Optimum existiert: Inferenz bevorzugt 1:1, Training 1:4 (Korollar 1).
- Der Algorithmus ist asymptotisch optimal unter SETH (Satz 3).
- Die CPU-Nachfrage wächst im 1:1-Szenario bis 2032 um Faktor ~ 8 stärker als im 1:4-Szenario.



plots/plot5_forecast_complexity.pdf

Abbildung 5: Links: CPU-Nachfrage-Prognose (normiert) 2022–2032 nach Szenario. Rechts: Gesamt-Komplexität $\Theta(n_T^3)$ im log-log-Diagramm. 1:1-Systeme skalieren günstiger als 1:8-Systeme um Faktor ~ 91 .

8 Ausblick

Der folgende Ausblick skizziert die zu erwartenden Entwicklungen in den Bereichen Hardware-Architektur, Algorithmen, Software-Infrastruktur, Ökonomie und gesellschaftliche Konsequenzen für den Zeitraum 2027–2032.

8.1 Hardware-Architektur: Weiterentwicklung der CPU:GPU-Integration

8.1.1 Heterogene Compute-Fabric

Die Trennung zwischen CPU und GPU wird in den nächsten Jahren zunehmend verwischen. Nvidia hat mit der Grace-Blackwell-Architektur (GB200) bereits demonstriert, dass CPU und GPU über NVLink-C2C mit 900 GB/s bidirektionaler Bandbreite verbun-

den werden können – ein Wert, der PCIe 5.0 (128 GB/s) um mehr als eine Größenordnung übertrifft [3]. Die nächste Generation – voraussichtlich Vera Rubin (VR200, erwartet 2027) – wird diesen Interconnect weiter ausbauen.

AMD verfolgt mit dem Venice-Prozessor und dem Infinity-Fabric-Interconnect eine ähnliche Strategie [1]. Für 2027 ist ein Infinity-Fabric-Gen5 angekündigt, der HBM4-Speicher direkt in die CPU-Die-Hierarchie einbettet.

8.1.2 3D-Stacked Compute

Ein langfristiger Trend ist die vertikale Integration von CPU, HBM-Speicher und NPU-Beschleunigern in einem einzigen Package. TSMC's SoIC-Technologie (System-on-Integrated-Chips) und Samsung's X-Cube ermöglichen ab 2028 Die-Stacking mit unter $1\ \mu\text{m}$ Pitch. Dies macht die konzeptuelle Trennung von CPU und GPU zunehmend obsolet – stattdessen entstehen *heterogene Rechen-Tiles*, die je nach Workload dynamisch als CPU-Tile oder GPU-Tile konfiguriert werden können [7].

8.1.3 In-Memory Computing und CXL

Compute Express Link (CXL) 3.1 (erwartet 2027) ermöglicht kohärenten Speicherzugriff über Rackgrenzen hinweg. In Kombination mit Processing-in-Memory (PIM)-Modulen (z. B. Samsung HBM-PIM, SK Hynix AiMX) können einfache Matrix-Operationen direkt im DRAM ausgeführt werden. Dies verschiebt die optimale Verhältnishierarchie weiter in Richtung 1:1 oder sogar 2:1 (mehr CPUs als GPUs), da CPU-nahe PIM-Module die Datenvorverarbeitung übernehmen [8].

8.2 Algorithmenentwicklung: Erweiterungen des Subgraph Algorithmus

8.2.1 Parallele Subgraph-Isomorphismus-Erkennung

Der Subgraph Algorithmus in seiner aktuellen Form [2] ist sequenziell mit $\Theta(n^3)$. Für sehr große Rack-Topologien ($n > 10\,000$ Knoten, z. B. NVL72-Cluster mit 72 GPUs pro Einheit mal tausende Einheiten) ist eine parallele Variante notwendig. Folgende Ansätze sind vielversprechend:

- **GPU-parallele Rotation:** Die n LCS-Berechnungen sind unabhängig (Lemma 1) und können auf n GPU-Threads verteilt werden, was die Laufzeit auf $O(n^2/p)$ reduziert (p = Anzahl paralleler Einheiten).
- **Approximative LCS:** Für Echtzeit-Topologie-Monitoring genügt ein Approximate-LCS mit Fehlerschranke ε , erreichbar in $O(n \log n)$ mittels Suffix-Array-Technik [6].
- **Streaming-Variante:** Bei dynamisch wechselnden Topologien (Hot-Plug von GPUs/C-PUs) können inkrementelle Updates der Signatur-Arrays in $O(n)$ statt $O(n^2)$ durchgeführt werden, sofern nur einzelne Knoten/Kanten geändert werden.

8.2.2 Gewichtete Topologien

Die aktuelle Definition des Hardware-Topologiegraphen (Definition 5) ist ungewichtet. Für präzisere Modelle sollten Kanten-Gewichte die Bandbreite des Interconnects kodieren:

$$w : E_r \rightarrow \mathbb{R}_{>0}, \quad w(c_i, \gamma_j) = \text{PCIe/NVLink-Bandbreite in GB/s}$$

Der Subgraph Algorithmus kann auf gewichtete Graphen erweitert werden, indem die Signatur-Funktion um einen Gewichtsterm ergänzt wird:

$$\sigma_j^w = \sum_{i=0}^{n-1} A_{ij} \cdot w(v_i, v_j) \cdot 2^i + j \cdot 2^n$$

Dies erfordert jedoch angepasste LCS-Algorithmen für reellwertige Alphabete.

8.2.3 Temporale Graphen

KI-Cluster sind dynamisch: GPUs werden hinzugefügt, entfernt oder aufgeteilt (Multi-Instance GPU, MIG). Temporale Graphen $G_r(t)$, bei denen $V(t)$ und $E(t)$ zeitabhängig sind, ermöglichen die formale Analyse von Cluster-Rebalancing-Strategien. Der Subgraph Algorithmus kann hier als *Snapshot-Vergleichen* eingesetzt werden, um zu entscheiden, ob die aktuelle Topologie $G_r(t_2)$ einen Subgraph der früheren Topologie $G_r(t_1)$ darstellt – und damit ob Konfigurationen verloren gegangen sind.

8.3 Software-Infrastruktur und KI-Frameworks

8.3.1 Topology-Aware Scheduling

Moderne KI-Frameworks (PyTorch 2.x, JAX, TensorFlow 2.x) sind noch weitgehend agnostisch gegenüber der physischen Hardware-Topologie. Eine Integration des Subgraph Algorithmus in den Scheduler würde ermöglichen, Tensor-Parallel-Gruppen automatisch an die optimale Sub-Topologie anzupassen [5]. Konkret:

- Erkenne, ob die aktuelle GPU-Gruppe eine 1:1- oder 1:2-Substruktur hat.
- Wähle Pipeline-Parallelismus-Tiefe entsprechend der Topologie-Hierarchie.
- Minimiere All-Reduce-Kommunikation durch topologie-bewusstes Tensor-Partitioning.

8.3.2 Automatische Hardware-Provisioning

Cloud-Anbieter (AWS, Azure, GCP) bieten zunehmend *Hardware-as-a-Service* mit variablen CPU:GPU-Verhältnissen. Ein Subgraph Algorithmus-basierter Provisioning-Agent könnte automatisch:

1. Den Workload-Graphen analysieren (Inferenz vs. Training vs. Feinabstimmung).
2. Die optimale Topologie G_r^* berechnen.
3. Den nächst-verfügbaren Superknoten als Subgraph $G_r^* \subseteq G_{\text{cluster}}$ identifizieren.
4. Die Ressourcen-Allokation entsprechend initiieren.

8.3.3 KI-Beschleuniger jenseits von GPU und CPU

Die Diskussion CPU vs. GPU wird ab 2027 durch weitere Akteure ergänzt:

- **NPU (Neural Processing Units):** Apple M-Serie, Qualcomm Hexagon, Intel Lunar Lake integrieren dedizierte NPUs. Im Subgraph-Modell erscheinen NPUs als neue Knotenklasse ν , was die Topologiedefinition auf $V_r = C_r \cup \Gamma_r \cup N_r$ (NPU-Knoten) erweitert.

- **Photonische Interconnects:** Lightmatter Passage und Ayar Labs entwickeln photonische On-Package-Interconnects mit > 10 Tbps Bandbreite. Dies ändert fundamental die Gewichtsfunktion $w(e)$ im gewichteten Topologiegraphen.
- **Neuromorphe Chips:** Intel Loihi 3 und IBM NorthPole bieten Spike-basierte Verarbeitung mit extremer Energieeffizienz für bestimmte Inferenz-Workloads. Die Integration in Subgraph-Topologien erfordert erweiterte Signatur-Funktionen für heterogene Knotentypen.

8.4 Ökonomische Konsequenzen

8.4.1 CPU-Markt: Renaissance und Wettbewerb

Die Verlagerung hin zu 1:1-Konfigurationen impliziert eine dramatische Erhöhung der CPU-Nachfrage in KI-Rechenzentren [1]. Konkret: Ein Rechenzentrum mit 100 000 GPUs in 1:8-Konfiguration benötigt 12 500 CPUs. Bei Umstieg auf 1:1 steigt der CPU-Bedarf auf 100 000 – Faktor 8. Dies begründet den erheblichen wirtschaftlichen Anreiz für AMD, die 1:1-Architektur aktiv zu bewerben [1].

Die wichtigsten Wettbewerber im Server-CPU-Segment für KI-Infrastruktur sind:

- **AMD Epyc Venice** (2026): Bis zu 192 Kerne, große L3-Cache-Kapazität, optimiert für KI-Prefill.
- **Nvidia Grace (Vera-Generation)** (2026–2027): ARM-basiert, tightly gekoppelt mit Blackwell/Rubin-GPUs via NVLink-C2C.
- **Intel Xeon Granite Rapids-D:** Hohe PCIe 5.0-Lane-Dichte für Storage-nahe KI-Workloads.
- **Amazon Graviton 5** (2026): Arm-basiert, optimiert für AWS-eigene KI-Dienste.
- **Google Axion** (2025/2026): Bespoke Arm-CPU für TPU-Integration.

8.4.2 Speicher: Engpass HBM und LPDDR6

Unabhängig vom CPU:GPU-Verhältnis bleibt Hochgeschwindigkeitsspeicher eine kritische Ressource [1]. HBM4 (erwartet 2026/2027) wird > 2 TB/s Bandbreite pro Stack bieten. Die Produktion ist jedoch durch TSMC-Kapazitäten limitiert. Für 1:1-Konfigurationen, die mehr CPUs benötigen, steigt auch der LPDDR6-Bedarf signifikant.

8.4.3 Energieverbrauch

Ein oft übersehener Aspekt: 1:1-Konfigurationen verbrauchen mehr Energie pro GPU-Rechenoperation, da die CPU-Leistungsaufnahme (~ 400 W für Epyc Venice) nicht durch mehr GPU-Arbeit amortisiert wird. Für Rechenzentren mit strengen PUE-Zielen ($< 1,2$) ist das 1:4-Format energetisch günstiger. Zukünftige Entwicklungen in der CPU-Effizienz (Chiplet-Designs, advanced node TSMC N2/A16) werden diesen Trade-off jedoch abschwächen.

8.5 Gesellschaftliche und wissenschaftliche Konsequenzen

8.5.1 Demokratisierung von KI-Inferenz

Die Verlagerung hin zu 1:1 senkt die Einstiegshürde für KI-Inferenz: Standard-CPU's (AMD Epyc, Intel Xeon) in Verbindung mit einzelnen High-End-GPU's (RTX 5090, Radeon RX 9900 XT) ermöglichen leistungsfähige lokale Inferenz. Dies fördert die Dezentralisierung von KI-Systemen und reduziert die Abhängigkeit von Cloud-Monopolen.

8.5.2 Wissenschaftliche Simulationen

Für wissenschaftliche HPC-Anwendungen (Klimamodellierung, Proteinfaltung, Quantenchemie) bietet das 1:1-Paradigma neue Möglichkeiten: CPU-intensive Vorverarbeitung und GPU-intensive Simulation können ohne Kommunikations-Overhead eng verzahnt werden. Dies ist insbesondere für Hybrid-Algorithmen relevant, die CPU-basierte symbolische Berechnungen (z. B. Differentialgleichungslöser) mit GPU-basiertem numerischen Computing kombinieren.

8.5.3 Formale Verifikation von KI-Infrastruktur

Der Subgraph Algorithmus bietet eine formale Grundlage für die *Infrastruktur-Verifikation*: Gegeben eine Deployment-Spezifikation als Graph G_{spec} , prüfe in $\Theta(n^3)$, ob die tatsächlich vorhandene Hardware-Topologie G_{real} die Spezifikation erfüllt ($G_{\text{spec}} \subseteq G_{\text{real}}$). Dies eröffnet neue Anwendungen im Bereich DevOps und Infrastructure-as-Code für KI-Cluster.

8.6 Offene Forschungsfragen

1. **Optimale Hybrid-Konfigurationen:** Existiert ein formales Optimierungsproblem, das CPU:GPU-Verhältnis, Workload-Profil und Energieverbrauch simultan minimiert? Ist dieses NP-hart?
2. **Dynamische Topologien:** Wie kann der Subgraph Algorithmus für dynamisch wechselnde Topologien (GPU-Hot-Plug, MIG) in Echtzeit betrieben werden?
3. **Optimalität ohne SETH:** Kann die $\Omega(n^3)$ -Schranke unbedingt (ohne Komplexitätshypothesen) bewiesen werden?
4. **Approximationsalgorithmen:** Welche Approximationsrate ist für Subgraph-Isomorphismus in Hardware-Topologien in $O(n^2 \log n)$ erreichbar?
5. **Quantencomputing:** Könnte ein Quantenalgorithmus (Grover-Suche, HHL) Subgraph-Isomorphismus in $O(n^{1.5})$ lösen und damit die SETH-Barriere umgehen?

Literatur

- [1] Rißka, V. (2026, 9. Mai). *Nach Nvidia, Meta & Co: Auch AMD bewirbt nun das 1:1-CPU-GPU-Verhältnis für AI.* ComputerBase. <https://www.computerbase.de/news/wirtschaft/nach-nvidia-meta-und-co-auch-amd-bewirbt-nun-das-1-1-cpu-gpu-verhaeltnis-fuer-ai-97281/> Abgerufen am 11.05.2026, 08:27 Uhr.

- [2] Epp, S. (2026). *Der Subgraph Algorithmus*. Universität Bielefeld. <https://gitlab.com/epp-group/subgraph>
- [3] NVIDIA Corporation (2024). *NVIDIA Grace Blackwell Superchip Architecture*. NVIDIA Technical White Paper. <https://resources.nvidia.com/en-us-blackwell-architecture>
- [4] Abboud, A., Backurs, A., & Vassilevska Williams, V. (2015). Tight Hardness Results for LCS and Other Sequence Similarity Measures. *Proceedings of FOCS 2015*, 59–78. <https://doi.org/10.1109/FOCS.2015.14>
- [5] Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., Phanishayee, A., & Zaharia, M. (2021). Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. *Proceedings of SC 2021*. <https://doi.org/10.1145/3458817.3476209>
- [6] Navarro, G. (2001). A guided tour to approximate string matching. *ACM Computing Surveys*, 33(1), 31–88. <https://doi.org/10.1145/375360.375365>
- [7] TSMC (2025). *System on Integrated Chips (SoIC) Technology Overview*. TSMC Technology Symposium 2025. <https://www.tsmc.com/technology/soic>
- [8] CXL Consortium (2024). *Compute Express Link Specification Revision 3.1*. <https://www.computeexpresslink.org/>
- [9] Meta AI Infrastructure Team (2025). *Meta Catalina: A 1:1 CPU-GPU Architecture for Large Language Model Inference*. *Hot Chips 2025*. <https://hotchips.org/2025/>
- [10] AMD (2026). *AMD EPYC Venice Processor Architecture Guide*. AMD Technical Documentation. <https://www.amd.com/en/processors/epyc>
- [11] Intel Corporation (2024). *Intel Xeon Scalable Processors for AI Workloads*. Intel Developer Zone. <https://www.intel.com/content/www/us/en/products/details/processors/xeon.html>
- [12] Patterson, D., & Hennessy, J. L. (2021). *Computer Organization and Design RISC-V Edition*. Morgan Kaufmann. ISBN 978-0-12-820331-6.
- [13] Jouppi, N. P., Hyun Yoon, D., Ashcraft, M., et al. (2023). TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. *Proceedings of ISCA 2023*. <https://doi.org/10.1145/3579371.3589350>
- [14] DeepSeek AI (2025). *DeepSeek-R2 Technical Report*. arXiv:2501.12948. <https://arxiv.org/abs/2501.12948>